

while 문

- while문의 기본 구조

```
while <조건문>:  
    <수행할 문장1>  
    <수행할 문장2>  
    <수행할 문장3>  
    ...
```

<초기값>

```
while <조건문>:  
    <수행할 문장>  
    ...  
<False값이 되는 조건문>
```

```
# 1~10 까지 숫자 + 메시지 출력  
cnt = 1 # 초기치  
while cnt <= 10: # 조건식  
    print(cnt, '=> Hello world')  
    cnt += 1 # cnt = cnt+1 # 증감치
```

while 문

```
# 10~1 숫자 출력
cnt = 10
while cnt > 0 :
    print(cnt, end=' ')
    cnt -= 1
```

```
# 1~100까지 누적합 구하기
# 1+2+3 ... + 100
cnt = 1
tot = 0
while cnt <= 100:
    tot += cnt
    cnt += 1
print(f'1~100까지의 합은 {tot}입니다. ')
```

퀴즈

입력받은 숫자로 구구단 출력하기

출력할 구구단의 숫자를 입력하세요...? 5

$$5 \times 1 = 5$$

$$5 \times 2 = 10$$

$$5 \times 3 = 15$$

$$5 \times 4 = 20$$

$$5 \times 5 = 25$$

$$5 \times 6 = 30$$

$$5 \times 7 = 35$$

$$5 \times 8 = 40$$

$$5 \times 9 = 45$$

퀴즈

별찍기1

```
1 *
2 **
3 ***
4 ****
5 *****
```

별찍기2

```
5 *****
4 *****
3 *****
2 *****
1 *****
```

사선출력

```
A
 B
  C
   D
    E
     F
      G
```

while 문 + if 조건문

```
# 1~50 사이의 숫자 중에서 짝수만 출력
cnt = 1
while cnt<=50:
    if cnt%2 == 0:
        print(cnt, end=' ')
    cnt += 1
```

```
# 1~100 사이의 숫자 중에서 3의 배수이거나 5의 배수 출력
cnt = 1
while cnt<=100:
    if (cnt%3 == 0) or (cnt%5 == 0):
        print(cnt, end=' ')
    cnt += 1
```

퀴즈

1~30 사이의 숫자를 아래와 같이 한줄에 5개씩 출력하여라

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25
26	27	28	29	30

퀴즈

리스트에서 평균을 구한 후 평균보다 큰 값만 출력하여라.

[100, 55, 50, 30, 25, 10, 67, 88, 45] 의 평균은? 52.22

평균보다 큰 값은? 100 55 67 88

다중 while 문

첫번째 while 문

cnt1 = 1

cnt2 = 1

while cnt1 <= 4:

print(cnt1, '*='*5)

두번째 while 문

cnt2 = 1

while cnt2 <= 3:

print('\t', cnt2, 'Hello world')

cnt2 += 1

cnt1 += 1

1 *==*==*==*==

1 Hello world

2 Hello world

3 Hello world

2 *==*==*==*==

1 Hello world

2 Hello world

3 Hello world

3 *==*==*==*==

1 Hello world

2 Hello world

3 Hello world

4 *==*==*==*==

1 Hello world

2 Hello world

3 Hello world

다중 while문 테스트 종료

퀴즈

다중 while문을 이용하여 2~9단 출력하기

2단

$$2 \times 1 = 2$$

..

$$2 \times 9 = 18$$

...

9단

$$9 \times 1 = 9$$

..

$$9 \times 9 = 81$$

무한루프 while

- 무한 루프

While True:

<수행할 문장1>

<수행할 문장2>

<수행할 문장3>

...

- 열 번 찍어 안 넘어 가는 나무 없다

```
treeHit = 0
```

```
while True:
```

```
    treeHit += 1
```

```
    print('나무를 %d번 찍었습니다.' % treeHit)
```

나무를 1번 찍었습니다.

나무를 2번 찍었습니다.

나무를 3번 찍었습니다.

...

break 문

- 명령문 실행시 제어문에서 탈출한다.
- 명령문이 실행되면 하단 명령문들은 실행되지 않는다.
- 무한루프의 종료 조건시 사용

```
while True값:
    명령문
    if 조건:
        break
```

q를 입력할때 까지 데이터 입력

```
while True:
```

```
    ans = input('데이터를 입력하세요(입력 종료는 q) ... ')
```

```
    if ans == 'q':
```

```
        break
```

```
print('break 테스트')
```

break 문

```
coffee = 10
money = '무한대'

while money:
    print("돈을 받았으니 커피를 줍니다.")
    coffee = coffee - 1
    print("남은 커피의 양은 %d개입니다." % coffee)
    print()

    # coffee 값이 0이면 while 문 탈출
    if coffee == 0:
        print("커피가 다 떨어졌습니다. 판매를 중지합니다.")
        break
```

break 문

q를 입력할때 까지 리스트에 데이터 추가

```
word_list = []
```

```
while True:
```

```
    data = input('데이터를 입력하세요(입력 종료는 q) ... ')
```

```
    if data == 'q':
```

```
        break
```

```
    else:
```

```
        word_list.append(data)
```

```
print(f'\n word_list = {word_list}')
```

```
데이터를 입력하세요(입력 종료는 q) ... 바나나
데이터를 입력하세요(입력 종료는 q) ... 비둘기
데이터를 입력하세요(입력 종료는 q) ... 파이썬
데이터를 입력하세요(입력 종료는 q) ... q
```

```
word_list = ['바나나', '비둘기', '파이썬']
```

continue문

- 하단의 명령문을 실행하지 않고 다음 명령문을 실행한다.
- 제어문을 빠져나가지는 않는다.

```
cnt = 0
while cnt<10:
    cnt += 1
    if cnt == 5:
        continue
    print(cnt, end=' ')
```

```
print('\n continue 테스트')
```

```
1 2 3 4 6 7 8 9 10
continue 테스트
```

pass 문

- 비실행문
- 제어문, 클래스, 함수 등에서 꼭 명령문이 들어가야 하지만 아무런 명령이 실행되게 하지 않고 싶을 때 사용한다.

```
pocket = ['paper', 'money', 'cellphone']
```

```
if 'money' in pocket:
```

```
    pass
```

```
else:
```

```
    print("카드를 꺼내라")
```

```
print('\n pass 테스트')
```

```
# 1~10 사이 숫자 중에서 5를 제외하고 나머지 출력하기
```

```
cnt = 0
```

```
while cnt<10:
```

```
    cnt += 1
```

```
    if cnt == 5:
```

```
        pass
```

```
    else:
```

```
        print(cnt, end=' ')
```

```
print('\n pass 테스트')
```

퀴즈

사용자가 정답을 맞힐 때까지 반복적으로 답을 입력할 수 있게 프로그래밍 하여라.

- while 문과 함께 break 또는 continue 활용
- 정답 키워드는 대문자, 소문자 구별없이 가능. upper(), lower() 함수 활용

퀴즈 : 파이썬에서 입력문 키워드는?

정답을 입력하세요: if

틀렸습니다. 다시 시도하세요.

정답을 입력하세요: input

정답입니다!

퀴즈 : 파이썬에서 입력문 키워드는?

정답을 입력하세요: in

틀렸습니다. 다시 시도하세요.

정답을 입력하세요: INPUT

정답입니다!

range() 함수

- `range(start, end , step)`
- `start` ~ `(end-1)`까지 `step`만큼 순차적으로 숫자 생성
- `range` 객체로 생성되므로 실제 출력을 확인하려면 리스트, 튜플, 집합 형태로 자료형을 변경하거나 `for`문과 함께 사용해야 한다.

`list( range(start, end , step) )` : 순차적으로 숫자로 구성된 리스트

`tuple( range(start, end , step) )` : 순차적으로 숫자로 구성된 튜플

`set( range(start, end , step) )` : 순차적으로 숫자로 구성된 집합

```
print(range(1, 10), type(range(1, 10)))
```

```
print(list(range(1, 10)))
```

```
print(tuple(range(1, 10)))
```

```
print(set(range(1, 10)))
```

```
print(list(range(1, 11)))
```

```
print(list(range(1, 20, 2)))
```

```
print(list(range(10, 0, -1)))
```

for 문과 range()

for 아이템변수 in range(start, end, step):

명령문

```
print('1~10까지 출력하기')
```

```
for i in range(1, 11):
```

```
    print(i, end=' ')
```

```
print('10~1까지 출력하기')
```

```
for i in range(10, 0, -1):
```

```
    print(i, end=' ')
```

```
print('1~100 까지 홀수만 출력하기')
```

```
for i in range(1, 101, 2):
```

```
    print(i, end=' ')
```

for 문과 range()

```
# for 문안에 if 문이 있는 형태
print('1~25 까지 한줄에 5개씩 출력하기 \n')
for i in range(1, 26):
    print(f'{i:<3}', end=' ')
    if i%5 == 0:
        print()
```

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

퀴즈

빈 리스트를 생성하고 데이터를 입력받아 리스트에 추가한다.

리스트에 삽입되는 데이터는 숫자이어야 한다.

음수의 숫자도 입력 대상이다

데이터 총 입력 횟수는 10번이며 리스트의 길이가 5이면 입력을 중단한다.

데이터를 입력하세요 ...?99
데이터가 추가되었음

데이터를 입력하세요 ...?-90
데이터가 추가되었음

데이터를 입력하세요 ...?python

데이터를 입력하세요 ...?파이썬

데이터를 입력하세요 ...?!^^

데이터를 입력하세요 ...?45
데이터가 추가되었음

데이터를 입력하세요 ...?0
데이터가 추가되었음

데이터를 입력하세요 ...?33
데이터가 추가되었음

```
numList = [99, -90, 45, 0, 33]
```

다중 for 문과 range()

for 아이템변수1 in range(start, end, step):

명령문1

for 아이템변수2 in range(start, end, step):

명령문2

```
for i in range(3): # 0, 1, 2
```

```
    print(i+1, '!='*5)
```

```
    for j in range(1, 4): # 1, 2, 3
```

```
        print('wt',j,'Hello world')
```

```
    print()
```

구구단 전체 출력

```
for i in range(2, 10): # 2~9
```

```
    print('=' * 30)
```

```
    print(f'Wn{i:^10}단Wn')
```

```
    for j in range(1, 10): # 1~9
```

```
        print(f'{i:^3} X {j:^3} = {i*j:^3}')
```

퀴즈

구구단을 아래와 같은 형태로 출력하여라

1 X 1 = 1	2 X 1 = 2	3 X 1 = 3
1 X 2 = 2	2 X 2 = 4	3 X 2 = 6
1 X 3 = 3	2 X 3 = 6	3 X 3 = 9
1 X 4 = 4	2 X 4 = 8	3 X 4 = 12
1 X 5 = 5	2 X 5 = 10	3 X 5 = 15
1 X 6 = 6	2 X 6 = 12	3 X 6 = 18
1 X 7 = 7	2 X 7 = 14	3 X 7 = 21
1 X 8 = 8	2 X 8 = 16	3 X 8 = 24
1 X 9 = 9	2 X 9 = 18	3 X 9 = 27
4 X 1 = 4	5 X 1 = 5	6 X 1 = 6
4 X 2 = 8	5 X 2 = 10	6 X 2 = 12
4 X 3 = 12	5 X 3 = 15	6 X 3 = 18
4 X 4 = 16	5 X 4 = 20	6 X 4 = 24
4 X 5 = 20	5 X 5 = 25	6 X 5 = 30
4 X 6 = 24	5 X 6 = 30	6 X 6 = 36
4 X 7 = 28	5 X 7 = 35	6 X 7 = 42
4 X 8 = 32	5 X 8 = 40	6 X 8 = 48
4 X 9 = 36	5 X 9 = 45	6 X 9 = 54
7 X 1 = 7	8 X 1 = 8	9 X 1 = 9
7 X 2 = 14	8 X 2 = 16	9 X 2 = 18
7 X 3 = 21	8 X 3 = 24	9 X 3 = 27
7 X 4 = 28	8 X 4 = 32	9 X 4 = 36
7 X 5 = 35	8 X 5 = 40	9 X 5 = 45
7 X 6 = 42	8 X 6 = 48	9 X 6 = 54
7 X 7 = 49	8 X 7 = 56	9 X 7 = 63
7 X 8 = 56	8 X 8 = 64	9 X 8 = 72
7 X 9 = 63	8 X 9 = 72	9 X 9 = 81

for ... in 문

for 변수 in 리스트 | 튜플 | 문자열 | 딕셔너리 :

명령문 ...

문자열 데이터 순회

myText = '가나다라마바사'

for item in myText:

print(f'** {item} **')

리스트 순회

cityList = ['구로동', '신림동', '서초동', '역삼동']

for item in cityList:

print(item)

튜플 순회

num_tuple = (10, -90, 100, 600, -300, -99, 50)

print(f'튜플 {num_tuple} 에서 음수만 출력')

for item in num_tuple:

if item < 0:

print(item, end=' ')

for ... in 문

리스트 순회 + if ~ else 조건문

```
marks = [90, 25, 67, 45, 80]
```

```
number = 0 for mark in marks:
```

```
    number = number + 1 # 증가
```

```
    if mark >= 60:
```

```
        print("%d번 학생은 합격입니다." % number)
```

```
    else:
```

```
        print("%d번 학생은 불합격입니다." % number)
```

1번 학생은 합격입니다.

2번 학생은 불합격입니다.

3번 학생은 합격입니다.

4번 학생은 불합격입니다.

5번 학생은 합격입니다.

for ... in 문

```
# 60점 이상인 데이터만 출력
```

```
# for 문 안에 if + continue
```

```
marks = [90, 25, 67, 45, 80]
```

1번 학생 축하합니다. 합격입니다.

3번 학생 축하합니다. 합격입니다.

5번 학생 축하합니다. 합격입니다.

```
number = 0
```

```
for mark in marks:
```

```
    number = number + 1
```

```
    if mark < 60: continue # continue 문인 경우 아래 명령 실행 X
```

```
    print("%d번 학생 축하합니다. 합격입니다. " % number)
```

for ... in 문

for 키변수 in 딕셔너리:

명령문

```
myDict = {1:'일', 100:'백', 50:'오십', 1000:'천'}
```

```
for key in myDict:
```

```
    print(f'키 : {key:<7} 값 : {myDict[key]}') # 딕셔너리명[키]
```

```
myDict = {1:'일', 100:'백', 50:'오십', 1000:'천'}
```

```
print(myDict.items())
```

```
for k, v in myDict.items():
```

```
    print(f'키 : {k:<7} 값 : {v}')
```

퀴즈

딕셔너리 값에 'a' 글자가 있는 아이템만 표시하고 총 갯수를 출력하여라.

```
word_dict = {'a': 'africa', 's': 'say', 'c': 'coffee', 'd': 'drama', 'y': 'yes'}
```

```
africa  
say  
drama  
총갯수는? 3
```

for ... in 문

- 다중 리스트와 for

리스트안에 삽입되어 있는 리스트의 갯수가 같아야 한다.

for (i, j...) in 다중리스트:

명령문

```
list_2d = [[1, 2], ['a', 'b'], ['홍길동', '춘향이']]
```

```
print(list_2d[0])
```

```
print(list_2d[0][0])
```

```
print(list_2d[2][1], list_2d[-1][-1])
```

```
[1, 2]
```

```
1
```

```
춘향이 춘향이
```

1행 1열 => 1

1행 2열 => 2

2행 1열 => a

2행 2열 => b

```
for i in range(3):
```

```
    print()
```

```
    for j in range(2):
```

```
        print(f'{i+1}행 {j+1}열 => {list_2d[i][j]}')
```

3행 1열 => 홍길동

3행 2열 => 춘향이

for ... in 문

```
listMulti2 = [[1, 2, 3, 4],  
              ['a', 'b', 'c', 'd'],  
              ['홍길동', '춘향이', '이몽룡', '고길동']]
```

```
for (i, j, k, l) in listMulti2:  
    print(f'i = {i:<8} j = {j:<5} k={k:<5} l={l:<5}')
```

i = 1	j = 2	k=3	l=4
i = a	j = b	k=c	l=d
i = 홍길동	j = 춘향이	k=이몽룡	l=고길동

퀴즈

아래의 리스트를 이용하여 grade 리스트를 생성하고
for 문을 이용하여 합계와 평균을 과목별로 출력한다.
평균은 소숫점 2번째 자리까지 출력한다.

```
kor = [100, 80, 85]
```

```
math = [55, 70, 35]
```

```
eng = [80, 80, 100]
```

```
python = [90, 70, 88]
```

```
grade => [[100, 80, 85], [55, 70, 35], [80, 80, 100], [90, 70, 88]]
```

```
kor : 합계 = 265 , 평균 = 88.33  
math : 합계 = 160 , 평균 = 53.33  
eng : 합계 = 260 , 평균 = 86.67  
python : 합계 = 248 , 평균 = 82.67
```

퀴즈

학생이름, 국어, 영어, 수학 으로 구성된 2차원 리스트를 생성하고
for 문을 이용하여 아래와 같이 출력하여라

```
stGradeList = [['김태희', 30, 50, 55],  
                ['신민아', 50, 90, 80],  
                ['박지민', 50, 90, 40],  
                ['김소희', 60, 50, 56],  
                ['윤준희', 90, 88, 66]]
```

학생이름	국어	영어	수학	합계	평균
김태희	30	50	55	135	45.00
신민아	50	90	80	220	73.33
박지민	50	90	40	180	60.00
김소희	60	50	56	166	55.33
윤준희	90	88	66	244	81.33

퀴즈

- 1) 학생 이름이 김으로 시작하는 데이터 만 출력하여라
- 2) 학생 이름이 희로 끝나는 데이터만 출력하여라

```
stGradeList = [['김태희', 30, 50, 55],  
               ['신민아', 50, 90, 80],  
               ['박지민', 50, 90, 40],  
               ['김소희', 60, 50, 56],  
               ['윤준희', 90, 88, 66]]
```

학생이름	국어	영어	수학
------	----	----	----

김태희	30	50	55
김소희	60	50	56

학생이름	국어	영어	수학
------	----	----	----

김태희	30	50	55
김소희	60	50	56
윤준희	90	88	66

리스트 내포

List comprehension

리스트안에 for 문이 있는 형태

리스트명 = [데이터 for 문]

for 문의 명령문에 의해 생성된 데이터가 리스트 안의 데이터로 추가된다

```
num_list = []  
for i in range(1,11):  
    num_list.append(i)  
print(num_list)
```

```
num_list2 = [i for i in range(1,11)]  
print(num_list2)
```

리스트 내포

```
# 3단의 결과값에서 -1 한 값으로 리스트 생성하기  
# 빈 리스트 생성 후 데이터 추가 방식
```

```
num_list1 = []  
for i in range(1,10):  
    num_list1.append((3*i)-1)  
print(num_list1)
```

```
# 리스트포방식  
num_list2 = [(3*i)-1 for i in range(1,10)]  
print(num_list2)
```

[2, 5, 8, 11, 14, 17, 20, 23, 26]

리스트 내포

빈 리스트 생성 후 데이터 추가 방식

```
star_list1 = []  
for i in range(1,11):  
    star_list1.append('*'*i)  
print(star_list1)
```

리스트포방식

```
star_list2 = ['*' * i for i in range(1,11)]  
print(star_list2)
```

```
['*', '**', '***', '****', '*****', '*****', '*****', '*****', '*****', '*****']
```

빈 리스트 생성 후 데이터 추가 방식

```
col_list1 = []  
for i in range(1,11):  
    col_list1.append(f'column{i}')  
print(col_list1)
```

리스트포방식

```
col_list2 = [ f'column{i}' for i in range(1,11)]  
print(col_list1)
```

리스트 내포

다중 리스트 내포 : 이중 for 문이 들어간 형태

리스트명 = [데이터 for ~ for ~]

구구단의 결과값으로 리스트 만들기

빈 리스트 생성 후 데이터 추가 방식

```
guguList1 = []
```

```
for i in range(2,10):
```

```
    for j in range(1,10):
```

```
        guguList1.append(i*j)
```

```
print(guguList1)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 3, 6, 9, 12, 15, 18, 21, 24, 27, 4, 8, 12, 16, 20, 24, 28, 32, 36, 5, 10, 15, 20, 25, 30, 35, 40, 45, 6, 12, 18, 24, 30, 36, 42, 48, 54, 7, 14, 21, 28, 35, 42, 49, 56, 63, 8, 16, 24, 32, 40, 48, 56, 64, 72, 9, 18, 27, 36, 45, 54, 63, 72, 81]
```

리스트 내포 방식

```
guguList2 = [i*j for i in range(2,10) for j in range(1,10)]
```

```
print(guguList2)
```

리스트 내포

조건문이 있는 리스트 내포 : 리스트명 = [데이터 for ~ if ~]

1~100 사이 숫자 중에서 3의 배수이거나 10의 배수로 구성된 리스트

빈 리스트 생성 후 데이터 추가 방식

```
myList1 = []
```

```
for i in range(1,101):
```

```
    if (i%3 == 0) or (i%10 == 0):
```

```
        myList1.append(i)
```

```
print(myList1)
```

리스트 내포 방식

```
myList2 = [ i for i in range(1,101) if i%3 == 0 or i%10 == 0 ]
```

```
print(myList2)
```

```
[3, 6, 9, 10, 12, 15, 18, 20, 21, 24, 27, 30, 33, 36, 39, 40, 42,
45, 48, 50, 51, 54, 57, 60, 63, 66, 69, 70, 72, 75, 78, 80, 81, 8
4, 87, 90, 93, 96, 99, 100]
```

리스트 내포

조건문 if ~ else 구문이 있는 리스트 내포

: 리스트명 = [데이터1 if ~ else 데이터2 for ~]

: 조건에 따라 데이터를 서로 다른 값으로 변환(Transformation)하여 채워 넣을 때는 if ~ else 구조가 반드시 for문 **앞쪽**에 배치

```
numbers = [1, 2, 3, 4, 5]
```

```
# 짝수는 '짝수', 홀수는 '홀수'라는 문자열로 변환하여 채우기
```

```
result = ["짝수" if x % 2 == 0 else "홀수" for x in numbers]
```

```
print(result)
```

```
['홀수', '짝수', '홀수', '짝수', '홀수']
```

퀴즈

다음과 같은 형태로 리스트를 리스트내포 방식을 이용하여 생성하여라

```
['row1-col1', 'row1-col2', 'row1-col3', 'row1-col4',  
'row2-col1', 'row2-col2', 'row2-col3', 'row2-col4']
```

퀴즈

다음과 같이 점수 리스트를 조건문이 있는 리스트 내포 방식을 이용하여 60점 이상은 'Pass', 60점 미만은 'Fail'로 변환하여라.

```
scores = [55, 80, 45, 92, 70]
```

```
출력: ['Fail', 'Pass', 'Fail', 'Pass', 'Pass']
```


함수 선언과 호출

- 함수란?
 - 함수(function) : 재사용 가능한 프로그램, 명령 덩어리
 - 함수(function)의 종류 : 내장함수, 사용자 정의 함수
 - 함수를 사용하는 이유는 사용하기 편하게 하기 위해서. 코드의 재사용

함수 선언과 호출

- 파이썬 함수의 구조

함수 정의

def 함수명(입력 인수):

 <수행할 문장1>

 <수행할 문장2>

...

def sum(a, b):

 result = a + b

 return result

함수 호출

함수명(입력 인수)

a = sum(3, 4)

print(a)

함수의 선언과 호출

- 함수의 종류

	Parameter 없음	Parameter 존재
반환 값 없음	함수 내의 수행문만 수행	인자를 사용, 수행문만 수행
반환 값 존재	인자없이, 수행문 수행 후 결과값 반환	인자를 사용하여 수행문 수행 후 결과값 반환

함수의 선언과 호출

- 인자가 없다. return이 없다.

함수 정의

```
def helloWorld():  
    print('Hello worldwtw*3')
```

함수 호출

```
helloWorld()
```

함수의 선언과 호출

- 인자가 있다. return이 없다.

```
def helloWorld2(user):  
    print(f'{user}님 안녕하세요! 오늘도 좋은하루')
```

함수 호출

helloWorld2('홍길동')

helloWorld2('고길동')

helloWorld2('박길동')

helloWorld2() # TypeError

홍길동님	안녕하세요!	오늘도	좋은하루
고길동님	안녕하세요!	오늘도	좋은하루
박길동님	안녕하세요!	오늘도	좋은하루

퀴즈

2개의 인자를 받아서 연산자(+,-,/,*,%)를 이용하여 연산결과 출력하기

함수 호출

cal(10,3)

10 + 3 = 13

10 - 3 = 7

10 * 3 = 30

10 / 3 = 3.33

10 % 3 = 1

퀴즈

숫자값을 인자로 전달받아서 출력하는 구구단 함수를 정의한 후 호출하여라

1	<code>gugu(9)</code>
---	----------------------

```
9 X 1 = 9
9 X 2 = 18
9 X 3 = 27
9 X 4 = 36
9 X 5 = 45
9 X 6 = 54
9 X 7 = 63
9 X 8 = 72
9 X 9 = 81
```

1	<code>gugu(12)</code>
---	-----------------------

```
12 X 1 = 12
12 X 2 = 24
12 X 3 = 36
12 X 4 = 48
12 X 5 = 60
12 X 6 = 72
12 X 7 = 84
12 X 8 = 96
12 X 9 = 108
```

퀴즈

숫자만큼 다음과 같은 형태로 별모양을 출력하는 함수를 정의한 후 호출하여라.

```
1 starPrint(10)
```

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * *
* * * * * * *
* * * * * * *
* * * * * * *
* * * * * * *
```

```
1 starPrint(5)
```

```
*
* *
* * *
* * * *
* * * * *
```


함수의 선언과 호출

- 인자가 없다. return문이 있다 => 반환값

return 계산|변수|데이터

return 문 아래아래의 명령은 수행되지 않는다

함수 정의

```
def classPrint():
```

```
    return 'MySQL, SQLite'
```

```
    print('!!!!!!!!!!!!')
```

함수호출

```
classPrint() # print()가 없어서 출력되지 않음
```

```
print('=====')
```

```
# print() 안에서 함수 호출
```

```
print(classPrint())
```

함수의 선언과 호출

함수 정의

```
def add_num():  
    n1 = input('첫번째 숫자 => ')  
    n2 = input('두번째 숫자 => ')  
    if n1.isdigit() and n2.isdigit():  
        return f'{n1} + {n2} = {int(n1)+int(n2)}'  
    else:  
        return '계산 불가능'
```

함수 호출1

```
print(add_num())
```

```
첫번째 숫자 => 23  
두번째 숫자 => 44  
23 + 44 = 67
```

함수 호출2

```
print(add_num())
```

```
첫번째 숫자 => 하나  
두번째 숫자 => 둘  
계산 불가능
```

함수의 선언과 호출

- 인자가 있다. return문이 있다

인사말 메시지 출력함수 정의

```
def messagePrint2(user):
```

```
    return 'Happy New Year.'+ user + ' 님'
```

함수 호출

```
print(messagePrint2('홍길동'))
```

```
print(messagePrint2('이영희'))
```

함수의 선언과 호출

1~n 까지의 누적합 구하기 함수 정의

```
def total(n):
```

```
    tot = 0
```

```
    for i in range(1, n+1):
```

```
        tot += i
```

```
    return tot
```

print() 안에서 함수 호출

```
print('1부터 50까지의 합은? ', total(50))
```

```
print('1부터 100까지의 합은? ', total(100))
```

함수의 선언과 호출

- return문의 값이 여러 개다.

```
return 값1, 값2 ...
```

- 튜플형태로 반환

함수 정의

```
def multiReturn(n, m):  
    return n+m, n-m
```

함수 호출

```
print(multiReturn(50, 20), type(multiReturn(50, 20)))
```

함수의 결과를 변수에 저장

```
result = multiReturn(50, 20)  
print(f'두수의 합은? {result[0]}')  
print(f'두수의 차는? {result[1]}')
```

함수의 선언과 호출

- 인자 모두에 초기값이 지정되어 있다

세수의 곱을 구하는 함수 정의

```
def multy(x=1, y=1, z=1):
```

```
    return f'{x} X {y} X {z} = {x*y*z}'
```

함수호출

```
print(multy())
```

```
print(multy(10))
```

```
print(multy(10, 20))
```

```
print(multy(10, 20, 30))
```

1 X 1 X 1 = 1

10 X 1 X 1 = 10

10 X 20 X 1 = 200

10 X 20 X 30 = 6000

함수의 선언과 호출

- 인자의 일부만 초기값이 설정되어 있다
주의 사항 : 초기값이 있는 인자는 맨뒤에 위치

함수 정의

```
def mul(n, m=1):  
    return print(f'{n} x {m} = {n*m}')
```

함수 호출

```
mul(7)  
mul(7, 8)  
mul() # 에러 발생 TypeError
```

아래는 에러발생 : SyntaxError

```
def mul_2(m=1, n):  
    return print(f'{n} x {m} = {n*m}')
```

퀴즈

아래의 함수를 호출하여 메시지가 출력되도록 함수를 정의하여라

함수 정의

```
def say_myself(name, old, man=True):
```

...

```
1 say_myself('김철수', 20)
```

나의 이름은 김철수 입니다.
나이는 20살입니다.
남자입니다.

```
1 say_myself('이민정', 15, False)
```

나의 이름은 이민정 입니다.
나이는 15살입니다.
여자입니다.

함수의 선언과 호출

- 가변인자 함수 *args
 - 가변인자(Variable-length)란 개수가 정해지지 않은 변수를 함수의 파라미터로 사용
 - Asterisk(*) 기호를 사용하여 함수의 파라미터를 표시한다.
 - 가변 인자는 입력 데이터값이 tuple 형태로 저장된다.
 - 일반 파라미터와 함께 사용할 경우에는 *args는 마지막에 위치해야 한다.

함수의 선언과 호출

함수 정의

```
def studentName(*args):  
    print(f'수강학생 목록 : {args}')  
    print(f'데이터형은? {type(args)}')  
    if len(args):  
        print(f'마지막 수강생 목록: {args[-1]}')
```

함수호출1 : 전달 데이터가 없다

```
studentName()
```

함수호출2 : 3개의 데이터값 전달

```
studentName('홍길동', '이순신', '이몽룡')
```

함수의 선언과 호출

총 갯수와 데이터를 번호와 함께 출력하는 함수 정의

```
def addList(*args):
```

```
    listResult = list(args)
```

```
    print(listResult, type(listResult))
```

```
    print(f'총 갯수 : {len(listResult)}')
```

```
    for i in range(0, len(listResult)):
```

```
        print(f'{i} 번째 => {listResult[i]}')
```

함수호출1

```
addList()
```

함수호출2

```
addList(1,2,3)
```

함수호출3

```
addList('가','나','다','라','마')
```

퀴즈

*args 를 이용하여 매개변수로 받은 학생 이름에 따라 아래와 같이 출력되도록 함수를 정의하여라

이때 매개변수가 없는 경우에는 '학생이 없습니다.' 메시지를 출력한다.

```
1 studentName2()
```

학생이 없습니다.

```
1 studentName2('홍길동')
```

1번째 학생 : 홍길동

```
1 studentName2('홍길동', '이순신', '이몽룡')
```

1번째 학생 : 홍길동

2번째 학생 : 이순신

3번째 학생 : 이몽룡

함수의 선언과 호출

- 일반인자와 가변인자가 함께 정의되어 있는 경우

주의사항 : 일반 인자는 앞에 배치, 가변인자는 맨 뒤에 배치

함수 정의 (인자, 가변인자) , return 문 있음

```
def printSymbolNumber(symbol, *args):
```

```
    return print(f'{symbol} {type(symbol)} // {args} {type(args)}')
```

함수 호출

```
printSymbolNumber('##')
```

```
printSymbolNumber('##', 10)
```

```
printSymbolNumber('$$$$', 10, 20, 30)
```

```
printSymbolNumber() # TypeError
```

```
## <class 'str'> // () <class 'tuple'>
```

```
## <class 'str'> // (10,) <class 'tuple'>
```

```
$$$$ <class 'str'> // (10, 20, 30) <class 'tuple'>
```

함수의 선언과 호출

```
# 아래는 함수 호출시 오류 발생 : TypeError
def printSymbolNumber2(*args , symbol):
    return print(f'{symbol} {type(symbol)} // {args} {type(args)}')

printSymbolNumber2('##')
printSymbolNumber2('##', 10)
printSymbolNumber2('$$$$', 10, 20, 30)
```

함수의 선언과 호출

계산기호인자 값에 따라서 뒤의 가변인자값을 계산하는 함수 정의

```
def calChoice(choice, *args) :
```

```
    if choice == '*':
```

```
        result = 1
```

```
        for i in range(0, len(args)):
```

```
            result *= args[i]
```

```
        return print(f'계산결과 = 곱 : {result}')
```

```
    elif choice == '+':
```

```
        result = 0
```

```
        for i in range(0, len(args)):
```

```
            result += args[i]
```

```
        return print(f'계산결과 = 합 : {result}')
```

```
    else:
```

```
        return print('계산 오류')
```

함수 호출

```
calChoice('*', 20,30)
```

```
calChoice('+', 20,30,50)
```

```
calChoice('!', 20,30,50)
```

계산결과 = 곱 : 600

계산결과 = 합 : 100

계산 오류

퀴즈

첫번째 인자의 값이 'min'이면 다음 인자의 숫자 중 최대값을 출력하고
첫번째 인자의 값이 'max'이면 다음 인자의 숫자 중 최소값을 출력하여라
이 때 전달하는 숫자의 갯수는 가변으로 한다.

```
1 min_max_number('min', 100, 20, 40)
```

최소값은? 20

```
1 min_max_number('max', 100, 20, 40, 500, 1)
```

최대값은? 500

```
1 min_max_number('plus', 100, 20, 40)
```

오류 발생

람다 함수

- 런타임에 생성해서 사용할 수 있는 익명 함수
- 쓰고 버리는 일시적인 함수. 보통 한줄로 간결하게 함수를 만들어 사용할 때 사용한다.
- def를 사용할 수 없는 곳에서 사용한다.
- 형식:
lambda 인자리스트: 표현식

람다 함수

일반함수 형태

```
def printWord(m):  
    return 'Message => ' + m
```

```
print(printWord('오늘도 좋은 하루'))
```

람다함수 형태

```
f = lambda message:('Message lambda => ' + message)  
print(f('오늘도 좋은 하루'))  
print(f('좋은 하루되세요'))
```

람다 함수

두수의 합을 구하는 일반함수

```
def print_sum(x, y):  
    return print(f'{x} + {y} = {x+y}')
```

print_sum(30, 100)

두수의 합을 구하는 람다함수 형태

```
f = lambda x,y:print(f'{x} + {y} = {x+y}')
```

f(30, 100)

퀴즈

첫 글자를 제외한 나머지 글자를 '*'로 표시하는 람다 함수를 정의하여라.

```
1 f1('홍길동')
2 f1('python')
3 f1('빅데이터인공지능')
4 f1('백합과장미')
```

```
홍**
p*****
빅*****
백****
```

퀴즈

국어,영어,수학 3과목의 합과 평균을 구하는 람다 함수를 정의하여라

```
1 f2(100, 80, 90)  
2 f2(77, 75, 56)
```

국어:100, 영어:80, 수학:90 , 총점:270, 평균:90.00

국어:77, 영어:75, 수학:56 , 총점:208, 평균:69.33

filter(), map()

- 정의된 사용자정의함수, 람다함수를 시퀀스 데이터 각각에 적용한다.
- filter(함수명/lambda 함수, 리스트/튜플)
사용할 함수는 결과값이 True/False
함수를 적용하여 시퀀스 데이터(튜플, 리스트등)에서 True 인 것만 추출한다
- map(함수명/lambda 함수, 리스트/튜플)
결과값을 시퀀스 데이터 요소로 추가한다.

filter()

숫자 리스트에서 짝수만 추출하여 새로운 리스트로 반환

filter()를 사용하지 않은 경우 일반 함수 적용

```
def evenNumber(list):
```

```
    resultList = []
```

```
    for i in list:
```

```
        if i%2 == 0:
```

```
            resultList.append(i)
```

```
    return resultList
```

```
print(f'짝수만 출력(일반함수) : {evenNumber([10, 30, 5, 9, 18])}')
```

filter()

숫자 리스트에서 짝수만 추출하여 새로운 리스트로 반환

일반함수 + filter()

짝수라면 True로 반환하는 함수 정의

```
def evenNumber2(n):
```

```
    return n%2 == 0
```

함수 호출

```
print(evenNumber2(10))
```

```
print(evenNumber2(5))
```

filter() 함수에 적용

```
print(filter(evenNumber2, [10, 30, 5, 9, 18]))
```

```
print(list(filter(evenNumber2, [10, 30, 5, 9, 18])))
```


filter()

숫자 리스트에서 짝수만 추출하여 새로운 리스트로 반환

람다함수 + filter()

짝수라면 True로 반환하는 람다함수

f3 = lambda n:n%2 == 0

함수 호출

print(f3(10))

print(f3(5))

필터에 적용후 리스트화

print(list(filter(lambda n:n%2 == 0, [10, 30, 5, 9, 18])))

filter()

```

numlist = [10, -30, 20, 5, -100]

# 양수인지 판독하는 함수 정의
def positive_num(list):
    result=[]
    for i in list:
        if i>0:
            result.append(i)
    return result

# 함수 호출
print(positive_num(numlist))

# filter() + 일반함수
def positive(n):
    return n>0

print(positive(-10))
print(positive(10))

print(list(filter(positive, numlist)))

# filter() + 람다함수
f_positive = lambda x:x>0

print(f_positive(-7))
print(f_positive(7))

print(list(filter(lambda x:x>0, numlist)))

```

퀴즈

`filter()` 함수를 이용하여 다음 리스트에서 글자 수가 4글자 이상인 단어만 골라 새로운 리스트로 생성하는 코드를 작성하여라.

```
words = ["apple", "it", "python", "ai", "banana", "go", "developer", "sql"]
```

```
['apple', 'python', 'banana', 'developer']
```

퀴즈

`filter()` 함수를 이용하여 다음 문자열에서 숫자만 리스트로 만들어 출력하여라. (`isdigit()` 이용)

```
message = 'ab4690cfvg342가1나1다0'
```

숫자만 출력

```
['4', '6', '9', '0', '3', '4', '2', '1', '1', '0']
```

map()

리스트의 제곱을 구해서 새로운 리스트로 만들기

```
numlist2 = [1,2,3,4]
```

```
# 제곱을 구하는 함수 정의
```

```
def power_fn1(mylist):
```

```
    result = []
```

```
    for i in mylist:
```

```
        result.append(i**2)
```

```
    return result
```

```
print(power_fn1(numlist2))
```

[1, 4, 9, 16]

```
# map() 사용할 제곱을 출력하는 함수 정의
```

```
def power_f2(value):
```

```
    return value**2
```

```
print(power_f2(3))
```

```
print(map(power_f2, numlist2))
```

```
print(list(map(power_f2, numlist2)))
```

```
# 람다함수 이용
```

```
f_multy = lambda x:x**2
```

```
print(f_multy(2))
```

```
print(list(map(lambda x:x**2, numlist2)))
```

map()

두 리스트에서 인덱스가 같은 값을 서로 곱한 후 리스트로 만들기

```
list1 = [2,3,7]
```

```
list2 = [4,5,9]
```

```
# 일반 함수
```

```
def multiply_n(list1, list2):
```

```
    resultList = []
```

```
    for i in range(0, len(list1)):
```

```
        resultList.append(list1[i]*list2[i])
```

```
    return resultList
```

```
print(list1, list2, '\n =>', multiply_n(list1, list2))
```

map()

```
list1 = [2,3,7]
```

```
list2 = [4,5,9]
```

```
# map() + 일반함수
```

```
def multiply(x,y):
```

```
    return x*y
```

```
print(list1, list2, '\n =>', list(map(multiply, list1, list2)))
```

```
# map() + 람다함수
```

```
f_multy2 = lambda x,y:x*y
```

```
print(f_multy2(2,4))
```

```
print(list1, list2, '\n =>', list(map(lambda x,y:x*y, list1, list2)))
```

퀴즈

4개의 리스트 각 아이템 요소가 '-'로 연결되어 출력되도록 하여라
최종 결과는 리스트로 저장되어야 하며 map() 함수를 활용한다

```
num_list = [100, 200, 300, 400]
```

```
name_list = ['길동', '동미', '미영', '영철']
```

```
gender_list = ['남', '여', '여', '남']
```

```
address_list = ['서울', '대전', '부산', '대구']
```

```
['100-길동-남-서울', '200-동미-여-대전', '300-미영-여-부산', '400-영철-남-대구']
```